

avito.tech 

ПОВЕДЕНЧЕСКИЕ ПАТТЕРНЫ

Афанасьев Юра

COMMAND



КОМАНДА

Инкапсулирует запрос как объект,
позволяя тем самым задавать
параметры клиентов для обработки
соответствующих запросов, ставить
запросы в очередь или
протоколировать их,
а также поддерживать
отмену операций

BEGIN;

UPDATE accounts **SET** balance = balance - 100.00 **WHERE** name = 'Alice';

UPDATE accounts **SET** balance = balance + 100.00 **WHERE** name = 'Bob';

COMMIT;

Отношение
как к транзакции

```
class Container
{
    public function process()
    {
        // действие 1
        // действие 2
        // действие 3

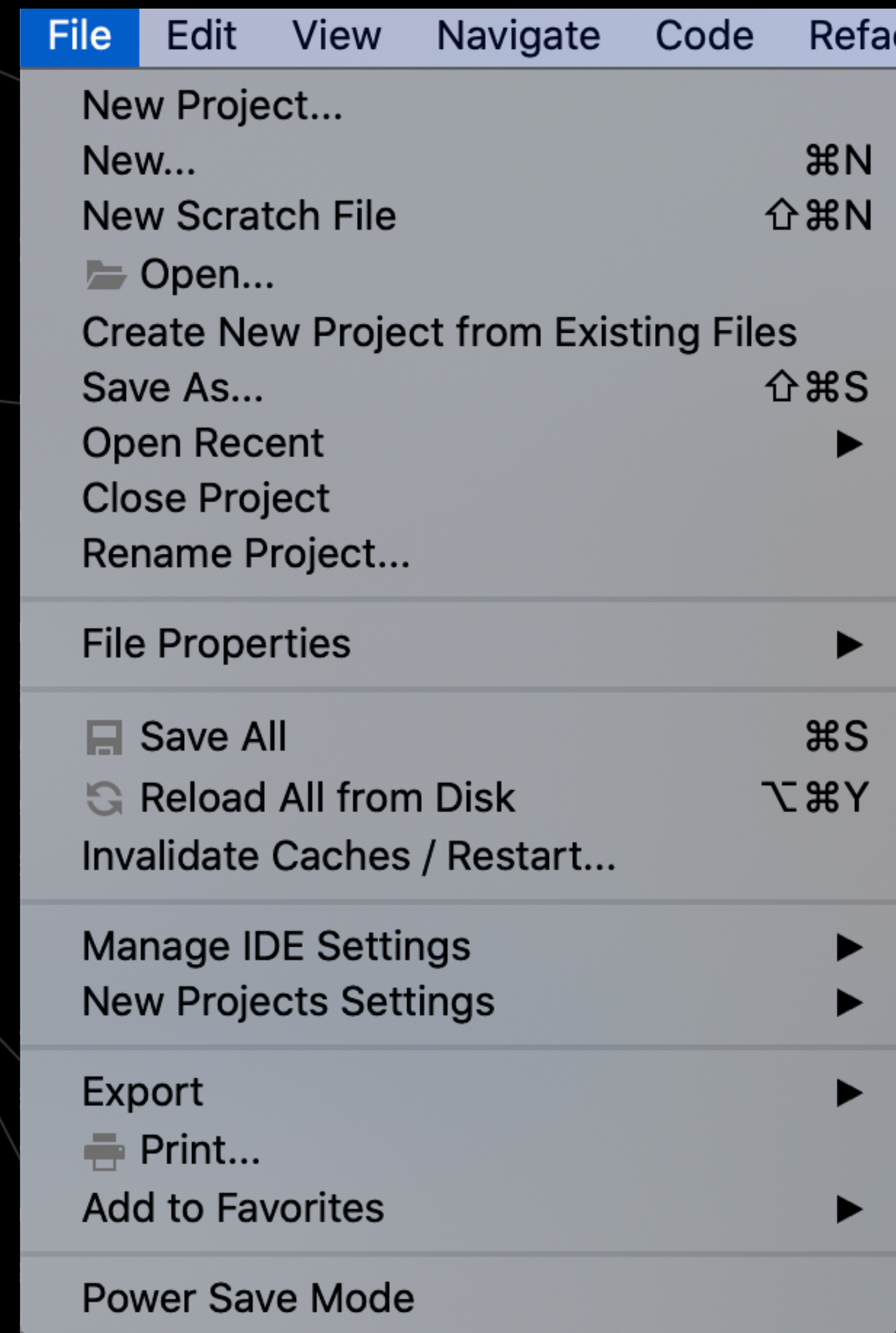
        // отношение к классу, как к контейнеру
        // для реализации любых операций
    }
}
```

Реализация
любых операций

Механика

- Логика работы с классом находится в команде
- Каждая команда инкапсулирует свой набор действий
- Все команды равнозначны друг относительно друга
- Клиент лишь выбирает нужную команду, ничего не зная о том, кто и как будет выполнять действие

Каждый пункт меню
может инкапсулировать
свою команду



Решаемая задача

- Разделяет инициатора действия и обработчики так, что они ничего не знают друг о друге
- Строит транзакции из операций
- Реализует операции отмены и повторного выполнения предыдущих действий
- Ведёт протокол всех действий



Получатель

Выполняет некоторую
бизнес-логику

```
class Receiver
{
    public function action()
    {
        echo 'Выполнение бизнес-логики.' . PHP_EOL;
    }

    // public function anotherAction() { }
}

// class AnotherReceiver {}
```

Методы получателей
вряд ли
пересекаются

Может содержать
много разных методов

Получателем
может выступать
любой класс в системе

Инициатор

Знает как отправить
запрос команде

Каждый метод выполняет
некоторое действие

Является чем-то
типа транзакции

```
class Invoker
{
    public function maker(Command $command)
    {
        // вызывает набор действий для работы с получателем
        $command->execute();
    }

    // public function anotherMaker(Command $command) {}
}
```

Каждый метод заточен
под своё действие

Интерфейс
команды

Задаёт необходимые
операции

Выполнение
запроса

Операция
отмены действия

Любые другие
действия

Например, операция
ведения протокола

```
interface Command
{
    public function execute();
    public function redo();

    // public function anotherAction();
}
```


Команда (знает
как взаимодействовать
с получателем)

```
class ConcreteCommand implements Command
```

```
{
```

```
    private Receiver $receiver;
```

Передаём
получателя

```
    public function __construct(Receiver $receiver)
```

```
    {
```

```
        $this->receiver = $receiver;
```

```
    }
```

```
    public function execute()
```

```
    {
```

```
        $this->receiver->action();
```

```
    }
```

Оперируем
получателем
для реализации задач

```
    public function redo()
```

```
    {
```

```
        echo 'Откат последней операции.' . PHP_EOL;
```

```
    }
```

Реализуем все
необходимые
действия

```
    // public function anotherAction() { }
```

```
}
```

```
// class AnotherConcreteCommand implements Command { }
```

На каждого получателя
создаётся своя команда

Каждый класс (получатель)
оборачиваем в свою команду

```
$receiver = new Receiver();  
$concreteCommand = new ConcreteCommand($receiver);  
testCommand($receiver);  
  
// $anotherReceiver = new AnotherReceiver();  
// $concreteCommand2 = new AnotherConcreteCommand($anotherReceiver);  
// testCommand($anotherReceiver);  
  
function testCommand(Command $concreteCommand)  
{  
    $invoker = new Invoker();  
    $invoker->maker($concreteCommand);  
    $invoker->redo($concreteCommand);  
}
```

Создаём необходимое
количество
получателей и команд

Передавая разные
команды, результат
будет отличаться

Инициатор обращается
к получателям так,
словно у них единый интерфейс

Особенности реализации

- Иницилирующий запрос объект располагает информацией о том, как его отправить, а не о том, как выполнить операцию
- Нужно заранее решить, насколько «умной» должна быть команда

Отделяет объект,
инициирующий
операцию,
от объекта,
который «знает»,
как её выполнить

MEDIATOR

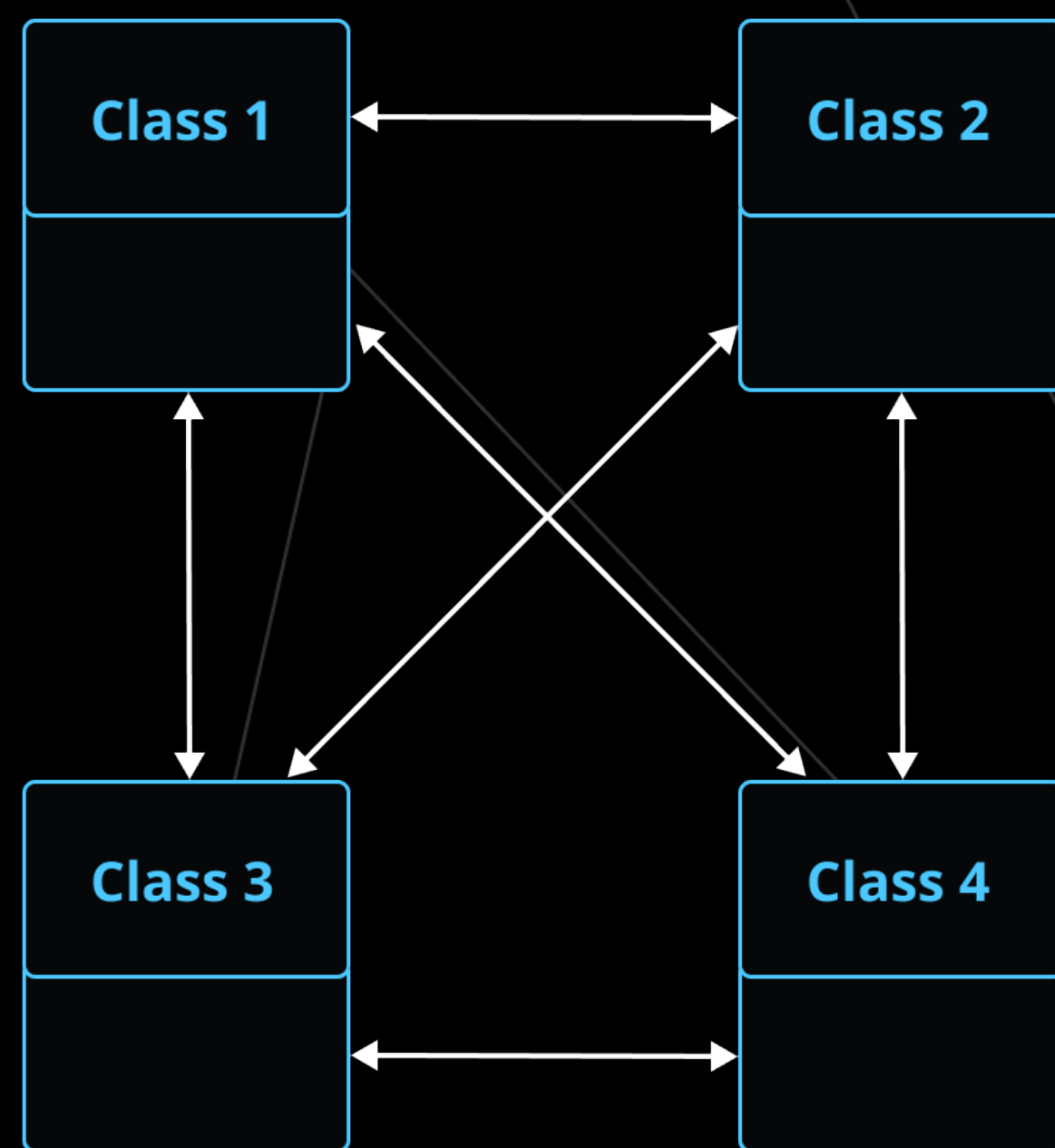


ПОСРЕДНИК

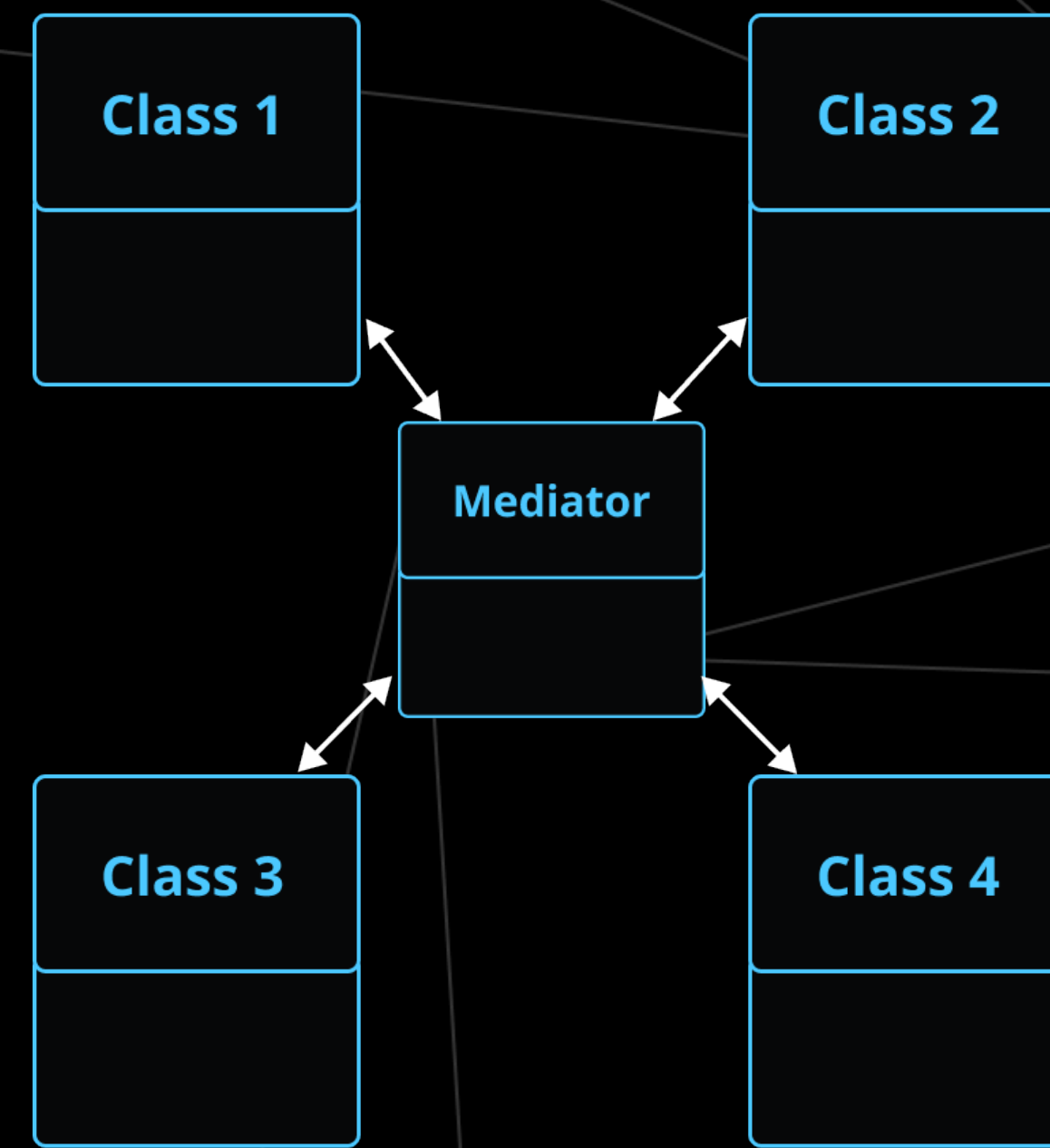
Определяет объект,
инкапсулирующий
способ взаимодействия
множества объектов

Обеспечивает слабую связанность
системы, избавляя объекты
от необходимости явно ссылаться
друг на друга, и позволяя тем самым
независимо изменять
взаимодействия между ними

Без паттерна

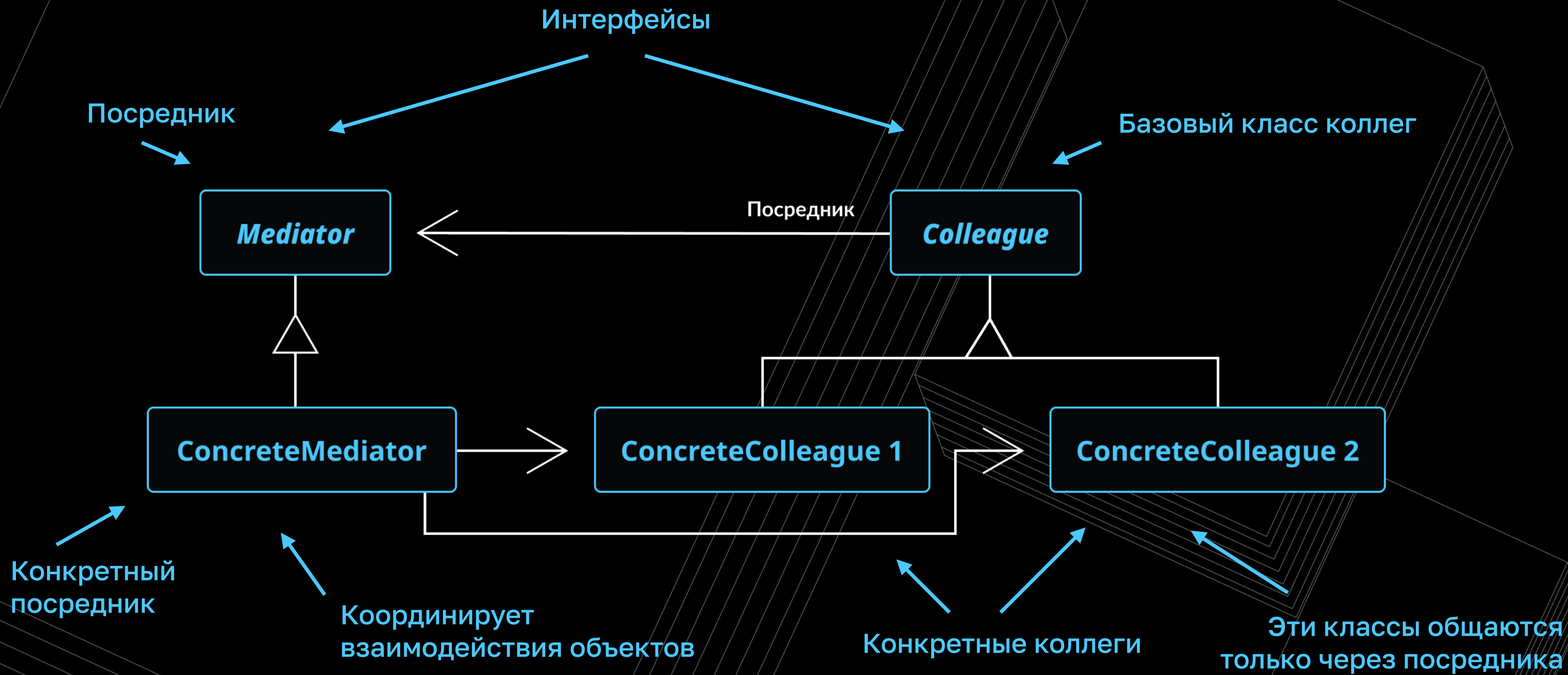


Используя паттерн



Решаемая задача

- Отвечает за координацию взаимодействий группы объектов
- Избавляет объекты от явных ссылок друг на друга
- Объекты знают только о посреднике, что сокращает количество взаимосвязей



Посредник

Метод для уведомления
о событиях

```
interface Mediator
{
    public function notify(object $sender, string $event): void;
}
```

Единственный
метод интерфейса

Конкретный
посредник

Передаём всех
коллег,
использующих
посредника

Конкретный
коллега,
отправляющий
событие

```
class ConcreteMediator implements Mediator
{
    public function __construct(
        private ConcreteColleague1 $concreteColleague1,
        private ConcreteColleague2 $concreteColleague2
        // private ConcreteColleague3 $concreteColleague3
    ) {
        $this->concreteColleague1->setMediator($this);
        $this->concreteColleague2->setMediator($this);
        // $this->concreteColleague2->setMediator($this);
    }

    public function notify(object $sender, string $event): void
    {
        if ($event === 'prepare') {
            echo 'Сработало событие на отправку данных.' . PHP_EOL;
            $this->concreteColleague1->sendData();
            $this->concreteColleague2->sendEmail();
        }

        if ($event === 'report') {
            echo 'Сработало событие на создание отчёта.' . PHP_EOL;
            $this->concreteColleague1->sendData();
        }

        // if ($event === 'another') {
        //     echo 'Сработало другое событие.' . PHP_EOL;
        //     $this->concreteColleague1->callSomeMethod();
        //     $this->concreteColleague2->callAnotherMethod();
        // }
    }
}
```

Управляемые
посредником
объекты
(коллеги)

Сообщаем всем
коллегам о себе

Событие

Каждое событие
реализует свои
действия

Обработчики
событий
создаём по
необходимости

Базовый
класс коллег

```
abstract class Colleague
{
    protected Mediator $mediator;

    public function setMediator(Mediator $mediator): void
    {
        $this->mediator = $mediator;
    }
}
```

Для взаимодействия
с другими коллегами
храним ссылку
на посредника

Коллеги

```
class ConcreteColleague1 extends Colleague
{
    public function prepare(): void
    {
        echo 'Подготовка данных.' . PHP_EOL;
        $this->mediator->notify($this, 'prepare');
    }

    public function sendData(): void
    {
        echo 'Отправка данных.' . PHP_EOL;
        $this->mediator->notify($this, 'sendData');
    }
}
```

Для взаимодействия
вызываем
посредника

```
class ConcreteColleague2 extends Colleague
{
    public function report(): void
    {
        echo 'Создание отчёта.' . PHP_EOL;
        $this->mediator->notify($this, 'report');
    }

    public function sendEmail(): void
    {
        echo 'Отправка e-mail.' . PHP_EOL;
        $this->mediator->notify($this, 'sendEmail');
    }
}
```

Коллег может
быть много

```
// class ConcreteColleague3 extends Colleague { }
```

Конфигурируем
посредника

```
function testMediator()
{
    $concreteColleague1 = new ConcreteColleague1();
    $concreteColleague2 = new ConcreteColleague2();
    $concreteMediator = new ConcreteMediator($concreteColleague1, $concreteColleague2);

    $concreteColleague1->prepare();

    echo PHP_EOL;

    $concreteColleague2->report();
}
```

Результат:
Подготовка данных.
Сработало событие
на отправку данных.
Отправка данных.
Отправка e-mail.

Результат:
Создание отчёта.
Сработало событие
на создание отчёта.
Отправка данных.

Работаем
с классами
как обычно

Плюсы

- Уменьшает связанность объектов (low coupling)
- Реализует концепцию «один ко многим»
- Централизует взаимодействие между объектами в системе
- Прост для понимания, сопровождения и расширения

Минусы



- Реализация может быть намного сложнее чем классы, использующие его
- Циклические ссылки между посредником и коллегами

Отвечает за
координацию
взаимодействий
между группой
объектов

Mediator

Facade

- Организует работу системы
- Предоставляет удобный интерфейс к системе

- Организует работу системы
- Обеспечивает связь между объектами

Mediator

Facade

- Абстрагирует систему объектов
- Связь всегда от фасада к классам системы

- Реализует обмен данных в системе
- Двухнаправленная связь между классами системы

avito.tech

Смотрите все серии на сайте
<https://avito.tech/patterns>

